TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



Asignatura: Inteligencia Artificial
Programa: Ingeniería Informática
Semestre: Noveno
Institución: UNIPAZ
Docente: Ing. Esp. Jhonis Ríos M.

INTRODUCCIÓN

En este taller práctico de la asignatura Inteligencia Artificial, los estudiantes de noveno semestre de Ingeniería Informática de UNIPAZ construirán, entrenarán y evaluarán una red neuronal utilizando **Python y las librerías TensorFlow, Keras, NumPy y scikit-learn.**

El propósito de la actividad es comprender cómo un modelo matemático puede convertirse en un programa capaz de aprender patrones a partir de datos. Los estudiantes identificarán la relación entre los modelos matemáticos, el Machine Learning, el Deep Learning y las redes neuronales.

En un ejercicio anterior, los pesos y sesgos de la red neuronal se definieron manualmente. En este taller, la red aprenderá automáticamente los valores de sus pesos y sesgos mediante un proceso de entrenamiento.

El caso de aplicación consiste en analizar conexiones informáticas utilizando dos características:

- Número de intentos fallidos de inicio de sesión.
- Cantidad de tráfico enviado, expresada en megabytes.

A partir de estas entradas, la red neuronal aprenderá a clasificar cada conexión como:

- **0: conexión normal.**
- **1: conexión sospechosa.**

Los datos utilizados serán simulados y tendrán una finalidad exclusivamente académica. Por esta razón, el programa no constituye un sistema real de detección de ataques informáticos.

Objetivo general: Construir, entrenar y evaluar una red neuronal de tres capas que permita clasificar conexiones informáticas simuladas como normales o sospechosas,



relacionando su funcionamiento con los modelos matemáticos, el Machine Learning y el Deep Learning.

Objetivos específicos

- Comprender la relación entre un modelo matemático y su implementación mediante código.
- Generar datos simulados para entrenar una red neuronal.
- Identificar las entradas, capas ocultas y salida del modelo.
- Comprender la función de los pesos y los sesgos.
- Aplicar las funciones de activación ReLU y sigmoide.
- Entrenar el modelo utilizando `modelo.fit()`.
- Evaluar el modelo mediante pérdida, exactitud, precisión y recall.
- Utilizar la red entrenada para clasificar datos nuevos.
- Diseñar una interfaz propia que permita interactuar con el modelo.

ARQUITECTURA DE LA RED NEURONAL

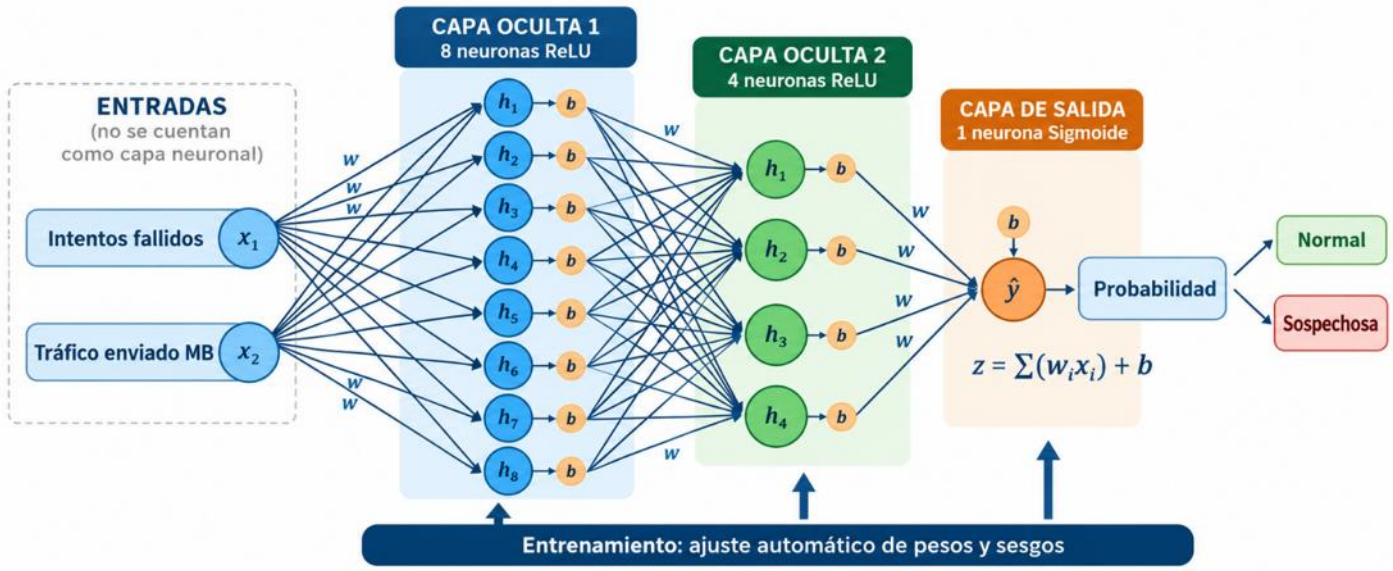


Figura 1. *Arquitectura de la red neuronal 2-8-4-1 utilizada para clasificar conexiones informáticas.*

La red neuronal tendrá una arquitectura **2-8-4-1**, compuesta de la siguiente manera:

- **Capa de entrada:** recibe dos valores.
- **Primera capa oculta:** contiene ocho neuronas con activación ReLU.



- **Segunda capa oculta:** contiene cuatro neuronas con activación ReLU.
- **Capa de salida:** contiene una neurona con activación sigmoide.

Las dos entradas del modelo serán:

- **x_1:** número de intentos fallidos de inicio de sesión.
- **x_2:** cantidad de tráfico enviado en megabytes.

La salida será una probabilidad comprendida entre 0 y 1.

- Una probabilidad inferior a 0,5 se interpretará como una conexión normal.
- Una probabilidad igual o superior a 0,5 se interpretará como una conexión sospechosa.

¿Qué hace el código?: El programa desarrolla las siguientes etapas:

1. Importación de librerías: Carga las herramientas necesarias para generar datos, dividirlos, normalizarlos, construir la red neuronal, entrenarla y evaluar sus resultados.

2. Configuración de la semilla: Establece un número fijo para que la generación de datos y el entrenamiento puedan repetirse y produzcan resultados similares.

3. Generación de datos: Crea conexiones simuladas con diferentes cantidades de intentos fallidos y tráfico enviado.

4. Creación de etiquetas: Asigna una respuesta correcta a cada conexión:

- `0` para una conexión normal.
- `1` para una conexión sospechosa.

5. División de los datos: Separa los datos en dos conjuntos:

- **Datos de entrenamiento:** se utilizan para que la red aprenda.
- **Datos de prueba:** se utilizan para comprobar cómo funciona la red con información que no observó durante el entrenamiento.

6. Normalización: Transforma las variables a una escala adecuada. Este procedimiento evita que el tráfico enviado, por tener valores numéricos mayores, domine el proceso de aprendizaje.



7. Construcción de la red neuronal: Define la arquitectura 2-8-4-1 mediante capas densas o `Dense`.

8. Compilación del modelo: Establece el optimizador, la función de pérdida y las métricas que se utilizarán durante el entrenamiento.

9. Entrenamiento: La red procesa los ejemplos, calcula sus errores y modifica automáticamente sus pesos y sesgos.

10. Evaluación: Mide el desempeño del modelo utilizando los datos de prueba.

11. Predicción: Permite introducir una conexión nueva para obtener una probabilidad y una clasificación.

¿Dónde está el cerebro matemático del código?: El cerebro matemático se encuentra principalmente en la construcción y el entrenamiento de la red neuronal.

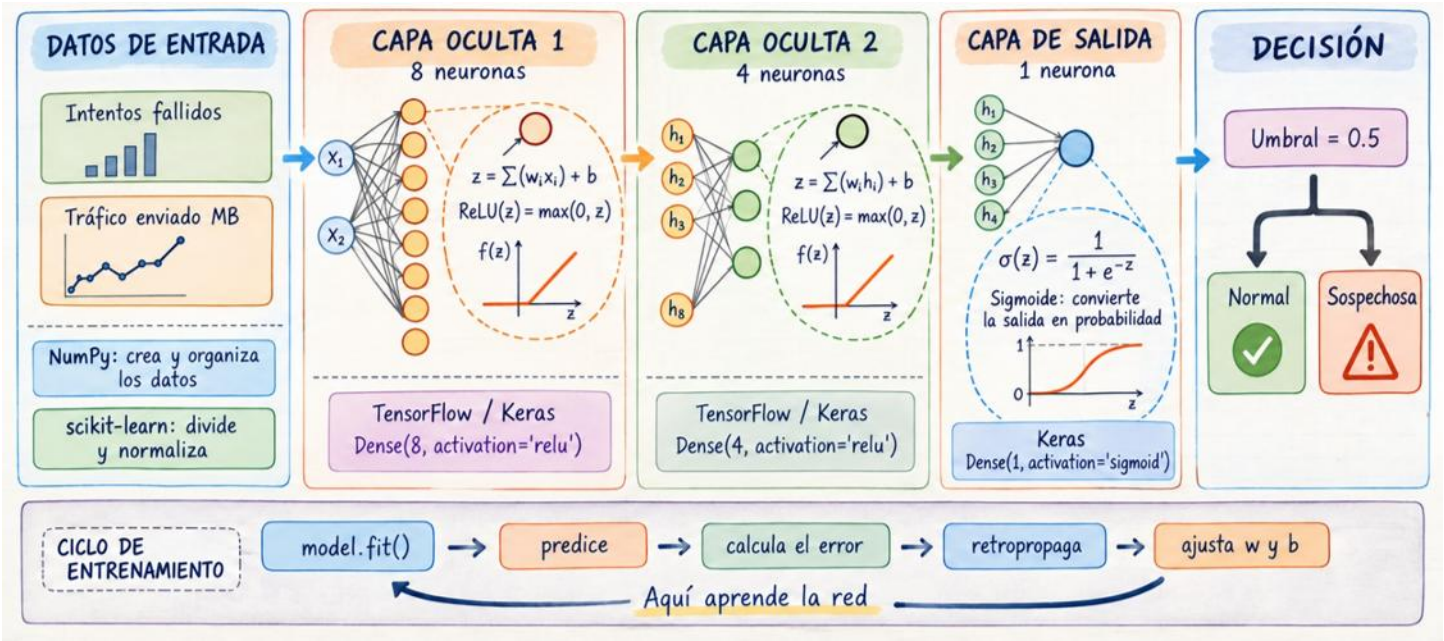


Figura 2. *Recorrido de la información y operaciones matemáticas realizadas por la red neuronal.*

Cada neurona recibe los datos de entrada, los multiplica por unos pesos, agrega un sesgo y aplica una función de activación.

TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



Combinación lineal: Cada neurona realiza inicialmente la siguiente operación:

$$z = (w_1 \times x_1) + (w_2 \times x_2) + b$$

En forma general: $z = \Sigma(w_i \times x_i) + b$

Donde:

- x_i**:** valores de entrada.
- w_i**:** pesos asignados a las entradas.
- **b:** sesgo de la neurona.
- **z:** resultado de la combinación lineal.

Los pesos indican la importancia que tiene cada entrada para la decisión del modelo. El sesgo permite ajustar la respuesta de la neurona.

Función de activación ReLU: Las neuronas de las capas ocultas utilizan la función ReLU: **ReLU(z) = máximo(0, z)**

Esto significa:

- Si z es menor que cero, la salida será cero.
- Si z es mayor que cero, la salida conservará el valor de z.

De forma simplificada: **Si z < 0, entonces ReLU(z) = 0.**

Si z ≥ 0, entonces ReLU(z) = z.

La función ReLU ayuda a la red neuronal a aprender relaciones que no pueden representarse únicamente mediante operaciones lineales.

Función sigmoide: La neurona de salida utiliza la función sigmoide:

$$p = 1 \div (1 + e^{-z})$$

La sigmoide transforma el resultado en un número comprendido entre 0 y 1. Este valor se interpreta como una probabilidad.



Por ejemplo:

- Si `p = 0,20`, existe una probabilidad del 20 % de que la conexión sea sospechosa.
- Si `p = 0,85`, existe una probabilidad del 85 % de que la conexión sea sospechosa.

La regla de clasificación será: **Si p ≥ 0,5, la conexión se clasifica como sospechosa.**

Si p < 0,5, la conexión se clasifica como normal.

¿Dónde ocurre el aprendizaje?: El aprendizaje ocurre principalmente cuando se ejecuta: `modelo.fit()`

Durante esta instrucción, la red realiza repetidamente los siguientes procesos:

1. Recibe los datos de entrada.
2. Realiza la propagación hacia adelante.
3. Calcula una predicción.
4. Compara la predicción con la etiqueta correcta.
5. Calcula la pérdida o error.
6. Propaga el error hacia atrás.
7. Actualiza los pesos y sesgos.
8. Repite el proceso durante varias épocas.

Este procedimiento se conoce como **entrenamiento de la red neuronal**.

DICCIONARIO BÁSICO DE REDES NEURONALES

Datos de entrada: Valores que recibe la red neuronal para realizar una predicción. En este ejercicio son los intentos fallidos y el tráfico enviado.

Etiquetas: Respuestas correctas que se entregan al modelo para que aprenda. En este ejercicio:

- `0` significa conexión normal.
- `1` significa conexión sospechosa.

Semilla: Número que permite repetir un experimento y obtener resultados similares cada vez que se ejecuta el programa.

TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



La semilla no es un dato que se utiliza para clasificar conexiones. Su función consiste en controlar la generación aleatoria de datos y facilitar la comparación de los resultados.

Pesos w: Valores que indican la importancia de cada dato de entrada. La red neuronal ajusta automáticamente los pesos durante el entrenamiento.

Sesgos b: Valores adicionales que ayudan a las neuronas a ajustar sus respuestas, incluso cuando las entradas son pequeñas o iguales a cero.

Neuronas ReLU: Unidades de las capas ocultas que procesan información. Si reciben un resultado negativo, entregan cero; si reciben un resultado positivo, conservan ese valor.

Sigmoide: Función utilizada en la capa de salida para convertir el resultado de la red en un número entre 0 y 1.

Época: Una época corresponde a una revisión completa de los datos de entrenamiento por parte de la red neuronal.

Pérdida: Valor que representa el error cometido por el modelo. Durante el entrenamiento se espera que la pérdida disminuya.

`modelo.fit()`: Instrucción que inicia el entrenamiento. El modelo practica con los datos, compara sus respuestas con las etiquetas correctas y modifica sus pesos y sesgos.

Exactitud o accuracy: Porcentaje general de predicciones correctas realizadas por el modelo.

Precisión: Indica cuántas de las conexiones clasificadas como sospechosas realmente pertenecían a esa categoría.

Recall: Porcentaje de conexiones sospechosas reales que el modelo logró detectar. Un recall alto significa que el modelo dejó escapar pocos casos sospechosos.

Predicción: Resultado producido por el modelo cuando recibe datos nuevos.

Umbral: Valor utilizado para convertir una probabilidad en una clasificación. En este ejercicio se utilizará un umbral de 0,5.

Actividad práctica: Los estudiantes deberán completar y ejecutar el código estructurado suministrado por el docente.

TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



Fase 1. Preparación del entorno

1. Instalar Python.
2. Crear una carpeta para el proyecto.
3. Abrir la carpeta en Visual Studio Code.
4. Instalar las librerías solicitadas.
5. Comprobar que las librerías puedan importarse correctamente.

Fase 2. Comprensión de los datos

1. Identificar las dos variables de entrada.
2. Revisar cómo se generan los datos simulados.
3. Explicar cómo se crean las etiquetas.
4. Contar cuántas conexiones normales y sospechosas fueron generadas.

Fase 3. Construcción de la red neuronal

1. Crear una primera capa oculta con ocho neuronas ReLU.
2. Crear una segunda capa oculta con cuatro neuronas ReLU.
3. Crear una capa de salida con una neurona sigmoide.
4. Mostrar el resumen de la arquitectura.
5. Identificar la cantidad de parámetros entrenables.

Fase 4. Entrenamiento

1. Compilar el modelo.
2. Ejecutar `modelo.fit()`.
3. Observar la pérdida durante las épocas.
4. Explicar por qué los pesos y sesgos cambian durante el entrenamiento.
5. Determinar si el modelo está aprendiendo.

Fase 5. Evaluación

1. Evaluar el modelo con los datos de prueba.
2. Calcular la exactitud.
3. Calcular la precisión.
4. Calcular el recall.
5. Explicar con sus propias palabras el significado de cada métrica.



Fase 6. Predicción: Probar el modelo con diferentes conexiones nuevas.

Como mínimo, se deben probar los siguientes casos:

- Una conexión con pocos intentos fallidos y poco tráfico.
- Una conexión con valores intermedios.
- Una conexión con muchos intentos fallidos y mucho tráfico.

Para cada caso, el programa debe mostrar:

- Valores introducidos.
- Probabilidad calculada.
- Umbral utilizado.
- Clasificación final.
- Explicación básica del resultado.

Resultados esperados: Cuando termine la ejecución, los estudiantes observarán:

- La cantidad de conexiones normales y sospechosas generadas.
- La estructura de la red neuronal.
- El número de parámetros entrenables.
- El avance del entrenamiento por épocas.
- La pérdida obtenida por el modelo.
- La exactitud o porcentaje general de aciertos.
- La precisión de las alertas generadas.
- El recall de las conexiones sospechosas.
- La probabilidad calculada para cada conexión nueva.
- La clasificación final como normal o sospechosa.

Los porcentajes pueden variar ligeramente. Sin embargo, se espera que la pérdida disminuya durante el entrenamiento y que las métricas mejoren progresivamente.

Una exactitud elevada indica que el modelo aprendió razonablemente el patrón utilizado para construir los datos simulados. No significa que pueda detectar ataques reales.

Resultados de aprendizaje: Al finalizar el taller, los estudiantes estarán en capacidad de:



- Explicar la función de las entradas, los pesos y los sesgos.
- Identificar las capas de una red neuronal.
- Diferenciar la función ReLU de la función sigmoide.
- Explicar la diferencia entre entrenamiento y prueba.
- Reconocer dónde ocurre la retropropagación.
- Interpretar la pérdida, la exactitud, la precisión y el recall.
- Utilizar una red entrenada para clasificar datos nuevos.
- Comprender que un modelo didáctico no debe confundirse con un sistema real de seguridad.
- Modificar la arquitectura y observar cómo cambian los resultados.
- Conectar una red neuronal entrenada con una interfaz.

ANEXO 1

CÓDIGO ESTRUCTURADO PARA COMPLETAR

El docente entregará un código estructurado que contiene comentarios explicativos y espacios que los estudiantes deberán completar.

Los comentarios del código indicarán:

- Las librerías que deben instalar.
- Las librerías que deben importar.
- El lugar donde se configura la semilla.
- La generación de los datos.
- La creación de las etiquetas.
- La división de los datos.
- La normalización.
- La construcción de las capas.
- La compilación del modelo.
- La ejecución de `modelo.fit()`.
- La evaluación.
- La predicción de conexiones nuevas.

El código no debe entregarse completamente resuelto. Cada estudiante o grupo deberá analizar los comentarios, completar las instrucciones faltantes y explicar el funcionamiento del programa.

TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



ANEXO 2

RETO DE DISEÑO DE UNA INTERFAZ PROPIA

Ejemplo demostrativo de una interfaz para interactuar con la red neuronal

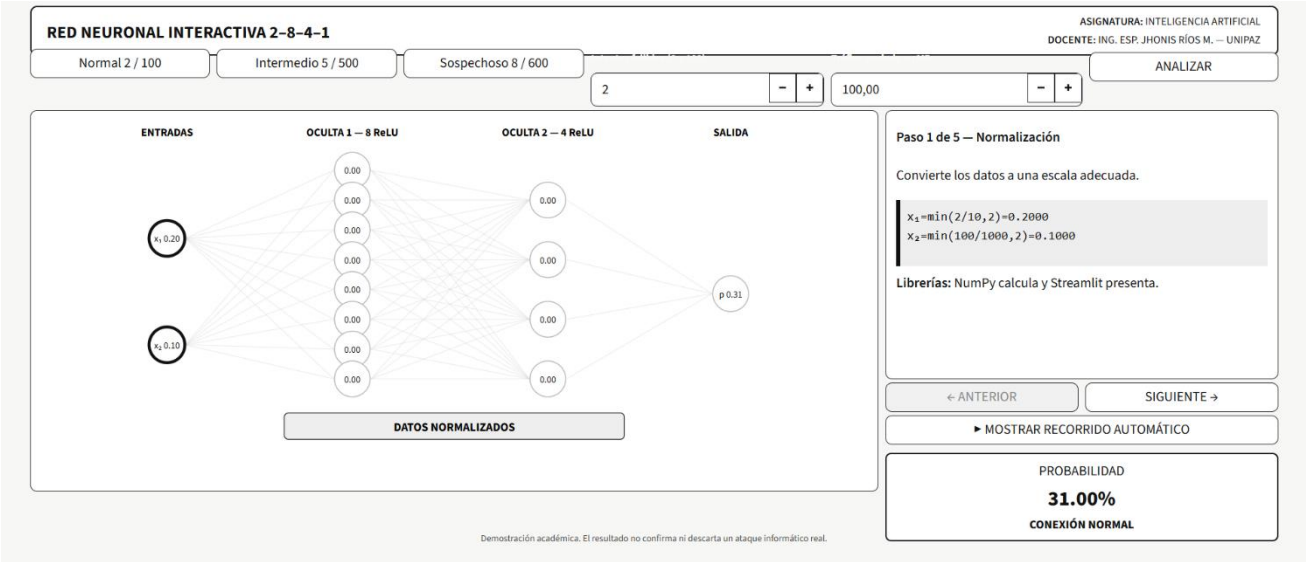


Figura 3. *Interfaz demostrativa del docente. Los estudiantes deberán desarrollar un diseño diferente y original.*

Después de completar y comprobar el funcionamiento de la red neuronal, cada grupo deberá diseñar una interfaz propia para interactuar con el modelo.

La interfaz mostrada por el docente tendrá una finalidad exclusivamente demostrativa. Los estudiantes no deberán copiar su diseño. Cada grupo deberá proponer su propia distribución, apariencia y forma de presentar los resultados.

Tecnologías permitidas

Los estudiantes podrán utilizar una de las siguientes opciones:

- Streamlit.
- Gradio.
- Flask con HTML y CSS.
- Django.
- FastAPI con HTML y CSS.
- PySide.



- PyQt.
- Otra tecnología autorizada por el docente.

No se exige utilizar una tecnología específica. Sin embargo, el grupo deberá explicar por qué seleccionó la herramienta empleada.

Requisitos mínimos de la interfaz

La interfaz deberá contener:

1. Nombre de la asignatura.
2. Título del proyecto.
3. Campo para introducir los intentos fallidos.
4. Campo para introducir el tráfico enviado en megabytes.
5. Botón para analizar la conexión.
6. Presentación de la probabilidad calculada.
7. Presentación del umbral utilizado.
8. Clasificación como normal o sospechosa.
9. Explicación básica del resultado.
10. Validación de los datos introducidos.
11. Mensaje que indique que se trata de una demostración académica.
12. Conexión real con la red neuronal entrenada.

No se aceptará una interfaz que muestre resultados fijos o inventados. Los resultados deberán ser generados por el modelo.

Entregables

Cada grupo deberá entregar:

- Código fuente de la red neuronal.
- Código fuente de la interfaz.
- Archivo `requirements.txt`.
- Instrucciones para instalar las librerías.
- Instrucciones para ejecutar el programa.
- Demostración del programa funcionando.
- Explicación de la arquitectura utilizada.
- Interpretación de las métricas.
- Explicación de cómo la interfaz se comunica con el modelo.

TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



Preguntas para el informe

1. ¿Cuáles son las entradas de la red neuronal?
2. ¿Qué representa cada etiqueta?
3. ¿Cuál es la función de los pesos?
4. ¿Cuál es la función de los sesgos?
5. ¿Por qué se normalizan los datos?
6. ¿Qué sucede dentro de una neurona?
7. ¿Qué función cumplen las capas ocultas?
8. ¿Por qué se utiliza ReLU en las capas ocultas?
9. ¿Por qué se utiliza sigmoide en la salida?
10. ¿Qué sucede cuando se ejecuta `modelo.fit()`?
11. ¿Qué diferencia existe entre exactitud, precisión y recall?
12. ¿Cómo se conectó la interfaz con el modelo?
13. ¿Qué dificultades se presentaron durante el desarrollo?
14. ¿Qué mejoras podrían aplicarse al proyecto?
15. ¿Por qué este modelo no debe utilizarse como un sistema real de seguridad?

Criterios de evaluación

- **Funcionamiento de la red neuronal: 30 %.**
- **Comprensión del modelo matemático: 20 %.**
- **Evaluación e interpretación de resultados: 15 %.**
- **Diseño y funcionamiento de la interfaz: 20 %.**
- **Organización y comentarios del código: 10 %.**
- **Presentación y sustentación: 5 %.**

Observación final

El objetivo principal del taller no es solamente ejecutar un código. Los estudiantes deberán comprender cómo los datos atraviesan las capas de la red, cómo se aplican las fórmulas matemáticas, cómo se ajustan los pesos y sesgos y cómo se produce una clasificación.

La interfaz permitirá observar el resultado de forma más comprensible, pero el componente central del proyecto continuará siendo la red neuronal y su proceso de aprendizaje.

TALLER PRÁCTICO

DE MODELOS MATEMÁTICOS, MACHINE LEARNING Y DEEP LEARNING CON REDES NEURONALES



Aviso académico: Los datos, probabilidades y clasificaciones producidos por este programa tienen fines educativos. El resultado no confirma ni descarta la existencia de un ataque informático real.